

Project: Data Engineering (DLMDSEDE02) — Final Abstract

Hakwoon Chung – IU14094574

1. Objectives and Concept

A dark factory is the manufacturing methodology of fully automating the production of goods at factories and other industrial facilities, without requiring any human labour presence on-site (Gisi, 2024). Such a facility produces a continuous stream of operational telemetry from warehouse management systems, high-rack storage sensors and automated equipment. This project designs and implements the data engineering backend for such a facility: a batch-processing infrastructure that ingests, stores, processes and serves operational telemetry to a machine learning application that retrains quarterly.

The architecture consists of six containerised microservices: Apache Kafka as a durable ingestion buffer, PostgreSQL holding both a raw append-only schema and an analytics schema of ML-ready features, Apache Spark for the quarterly aggregation, Apache Airflow orchestrating the recurring jobs, and an ELK stack providing centralised logging. A synthetic data generator expands two Kaggle reference datasets — Smart Logistics Supply Chain and High-Rack Storage System sensor data — into over one million timestamped rows, matching realistic factory schemas.

2. Implementation (making-of)

The stack was implemented in a Linux (Ubuntu) environment, orchestrated through a single Docker Compose file bringing up eleven services. A custom Airflow image adds OpenJDK 17, PySpark 3.5.3 and the Spark provider so that SparkSubmitOperator can submit jobs in client mode. The data generator builds per-column empirical distributions from the source CSVs and produces JSON events into Kafka in batches, using a seeded random generator for reproducibility. An Airflow DAG drains both topics into raw.* tables, then invokes a PySpark job that reads a single quarter, aggregates via groupBy, and writes idempotently into analytics.* by deleting and re-inserting the target quarter's rows. Container logs from application services are shipped to Logstash via Docker's GELF driver and indexed daily in Elasticsearch.

3. Reflection: requirements, issues, and extensions

The system fulfils the technical requirements set out in Phase 1. All eleven services reach a healthy state, the generator produces one million timestamped rows, the DAG completes end-to-end, and both analytics feature tables are populated as expected. Reproducibility is achieved through Infrastructure as Code — a single docker compose up command brings the environment online on any machine with Docker installed. There is unfortunately no easy fix for making applications reliable, scalable, or maintainable. However, there are certain patterns and techniques that keep reappearing in different kinds of applications. (Kleppmann, 2017, p.38 ~ 41): here, reliability rests on Kafka's disk-backed log, Airflow's retry semantics and idempotent Spark writes; scalability is expressed as a

pattern rather than a proof, since PySpark code runs unchanged from a single-node container to a multi-node cluster; and maintainability follows from microservice isolation and version control.

Three implementation issues stand out. First, Broadcom's migration of the free Bitnami images to a `bitnamilegacy/*` namespace broke initial image pulls; the fix was to pin legacy tags and adjust dependent healthchecks. Second, the Postgres JDBC driver required three attempts before bind-mounting the JAR from the host worked reliably. Third, the Spark master URL passed via `SparkSubmitOperator`'s connection did not survive parsing, so the master URL was hard coded in the `SparkSession` builder.

Data security, governance and protection are implemented at a baseline: credentials via environment variables excluded from version control, private Docker networks and no personal data in the synthetic dataset. Production extensions would add TLS between services, secrets management via HashiCorp Vault, per-service database roles with least-privilege, a data catalogue for lineage, and encryption at rest for Postgres and Elasticsearch volumes.

4. Extending toward a streaming pipeline

A second, real-time pipeline could be added alongside the batch one without displacing it. Kafka is already the ingestion buffer and would remain, but a streaming processor — Apache Flink or Spark Structured Streaming — would consume the same topics with a distinct consumer group, applying windowed aggregations (tumbling or sliding) and writing to a low-latency serving store such as Elasticsearch for dashboards. The batch and streaming layers would coexist in a Lambda-style topology: streaming provides fresh, approximate views while batch remains the authoritative source of truth. Airflow's role would narrow to periodic reconciliation and historical reprocessing.

5. Skills gained

Three technical skills are achieved: microservices orchestration in a container using the multi-service Docker Compose configuration file with healthchecks and custom images; data engineering plumbing between the raw and analytics layers, with idempotent transformations created without any upsert primitives; and orchestration and observability through Airflow combined with the ELK stack, both through the GELF driver of Docker. There are also three soft skills: scope management, where the temptation to include extra details like multi-broker Kafka, Kubernetes, and streaming was resisted because it would have taken away from the essence of the work done; honest self-reflection, where the issues encountered were documented honestly and not covered up; and structured decomposition, where the work was divided into clearly defined phases.

References

- Gisi, P. (2024). *The Dark Factory and the Future of Manufacturing: A Guide to Operational Efficiency and Competitiveness*. New York, NY: Routledge, Taylor & Francis Group. p. 3. doi:10.4324/9781032688152
- Kleppmann, M. (2017). *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. Heidelberg: O'Reilly. 1st Edition. p. 38–41, 488–490.